

Introduction

These four sessions cover the core pillars of Object-Oriented Programming (OOP) in Java. We will go **step by step**, explaining each concept with **clear theory, full code examples**, and **practice assignments** with detailed solutions.

By the end of these notes, you will have a strong foundation to write well-structured, maintainable Java code.

Session 8 – Encapsulation and Access Specifiers

1. Understanding Encapsulation

Encapsulation is the mechanism of **wrapping data (variables) and code (methods) together as a single unit** – the class – and **hiding the internal details** from the outside world.

In Java, we achieve encapsulation by:

- Declaring the fields (instance variables) as private – they cannot be accessed directly from outside the class.
- Providing public **getter and setter methods** to read and modify the fields, with optional validation.

Why Encapsulation?

- **Control** – you can add logic (validation, logging) inside setters.
 - **Read-only or write-only** fields – provide only a getter or only a setter.
 - **Maintainability** – internal changes do not affect external code as long as the public interface remains the same.
 - **Data hiding** – protects the integrity of data.
-

2. How Encapsulation is Achieved

Step 1: Make fields private.

Step 2: Provide public getter and setter methods (or only one if needed).

Step 3: (Optional) Add validation in setters.

Example:

```
java
class Person {
    // Private fields (data hiding)
    private String name;
    private int age;

    // Public constructor
    public Person(String name, int age) {
        setName(name); // reuse setter for validation
        setAge(age);
    }

    // Getter for name
```

```

public String getName() {
    return name;
}

// Setter for name with validation
public void setName(String name) {
    if (name != null && !name.trim().isEmpty()) {
        this.name = name;
    } else {
        System.out.println("Invalid name provided.");
    }
}

// Getter for age
public int getAge() {
    return age;
}

// Setter for age with validation
public void setAge(int age) {
    if (age >= 0 && age <= 150) {
        this.age = age;
    } else {
        System.out.println("Invalid age. Must be between 0 and 150.");
    }
}

// Additional method
public void display() {
    System.out.println("Name: " + name + ", Age: " + age);
}

}

public class Main {
    public static void main(String[] args) {
        Person p = new Person("Alice", 25);
        p.display();           // Name: Alice, Age: 25

        p.setAge(-5);          // Invalid age.
        p.setName("");          // Invalid name provided.
        p.display();           // Name: Alice, Age: 25 (unchanged)
    }
}

```

```
}  
}
```

Output:

text

Name: Alice, Age: 25

Invalid age. Must be between 0 and 150.

Invalid name provided.

Name: Alice, Age: 25

3. Access Specifiers (Access Modifiers)

Access specifiers define **who can access** a class, method, or field. Java provides four levels:

Modifier	Same Class	Same Package	Subclass (different package)	Anywhere (world)
private	✓ Yes	✗ No	✗ No	✗ No
default (no modifier)	✓ Yes	✓ Yes	✗ No	✗ No
protected	✓ Yes	✓ Yes	✓ Yes	✗ No
public	✓ Yes	✓ Yes	✓ Yes	✓ Yes

Explanation:

- private – accessible only inside the same class. Use this for fields to enforce encapsulation.
- default (also called package-private) – accessible only within the same package. No keyword is used.
- protected – accessible in the same package and also in subclasses (even if they are in different packages).
- public – accessible from anywhere.

Code Example Showing All

```
java  
package com.example;  
  
public class AccessExample {  
    private int privateVar = 1;  
    int defaultVar = 2;           // package-private  
    protected int protectedVar = 3;  
    public int publicVar = 4;  
}
```

```

private void privateMethod() {
    System.out.println("Private method");
}

void defaultMethod() {
    System.out.println("Default method");
}

protected void protectedMethod() {
    System.out.println("Protected method");
}

public void publicMethod() {
    System.out.println("Public method");
}

// Inside the same class – all accessible
public void test() {
    privateMethod(); // OK
    defaultMethod(); // OK
    protectedMethod(); // OK
    publicMethod(); // OK
}

// In another class in the same package
class TestSamePackage {
    public static void main(String[] args) {
        AccessExample obj = new AccessExample();
        // obj.privateVar; // ERROR
        // obj.privateMethod(); // ERROR
        System.out.println(obj.defaultVar); // OK (default)
        System.out.println(obj.protectedVar); // OK (protected within package)
        System.out.println(obj.publicVar); // OK
        obj.defaultMethod(); // OK
        obj.protectedMethod(); // OK
        obj.publicMethod(); // OK
    }
}

```

Note: For subclass in a different package, protected is accessible via inheritance (using super), but default is **not**.

4. Assignment 8

Task: Create a class BankAccount with private fields: accountNumber (String), balance (double). Provide public getters and setters for both fields. In the setter for balance, ensure that the new balance is not negative. Also add a method deposit(double amount) and withdraw(double amount) that use the setters (or directly modify the private field with validation). Write a main method to test all methods.

Solution 8

```
java
class BankAccount {
    private String accountNumber;
    private double balance;

    // Constructor
    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        setBalance(initialBalance); // use setter for validation
    }

    // Getter for accountNumber
    public String getAccountNumber() {
        return accountNumber;
    }

    // Setter for accountNumber (optional – we may not want to change it)
    public void setAccountNumber(String accountNumber) {
        if (accountNumber != null && !accountNumber.isEmpty()) {
            this.accountNumber = accountNumber;
        } else {
            System.out.println("Invalid account number.");
        }
    }

    // Getter for balance
    public double getBalance() {
        return balance;
    }

    // Setter for balance with validation (non-negative)
    public void setBalance(double balance) {
        if (balance >= 0) {
```

```

        this.balance = balance;
    } else {
        System.out.println("Balance cannot be negative.");
    }
}

// Deposit method
public void deposit(double amount) {
    if (amount > 0) {
        double newBalance = balance + amount;
        setBalance(newBalance); // reusing setter
        System.out.println("Deposited: " + amount);
    } else {
        System.out.println("Deposit amount must be positive.");
    }
}

// Withdraw method
public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        double newBalance = balance - amount;
        setBalance(newBalance);
        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("Insufficient balance or invalid amount.");
    }
}

public void display() {
    System.out.println("Account: " + accountNumber + ", Balance: $" + balance);
}

}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("A123", 1000);
        acc.display();           // Account: A123, Balance: $1000.0

        acc.deposit(500);        // Deposited: 500.0
        acc.display();           // Account: A123, Balance: $1500.0
    }
}

```

```
acc.withdraw(200);        // Withdrawn: 200.0
acc.display();           // Account: A123, Balance: $1300.0

acc.withdraw(2000);       // Insufficient balance or invalid amount.
acc.setBalance(-100);     // Balance cannot be negative.
acc.display();           // Account: A123, Balance: $1300.0
}
}
```

Session 9 – Inheritance

1. Understanding Inheritance

Inheritance is a mechanism where a **child class (subclass)** acquires the properties and behaviors of a **parent class (superclass)**.

It establishes an **IS-A** relationship: a Dog **is an** Animal.

In Java, inheritance is achieved using the `extends` keyword.

The subclass can:

- Use all non-private members of the parent.
- Add new members.
- Override methods of the parent.

Benefits:

- **Code reusability** – common code in parent.
- **Extensibility** – you can build upon existing classes.
- **Polymorphism** – enables runtime polymorphism (covered later).

Important: Java does **not** support multiple inheritance for classes (a class cannot extend more than one class) – but we can implement multiple interfaces.

2. How Inheritance is Achieved

Syntax:

```
class Child extends Parent { ... }
```

Example:

```
java
// Parent class
class Animal {
    String name;

    public Animal(String name) {
        this.name = name;
    }

    public void eat() {
        System.out.println(name + " is eating.");
    }
}
```

```

    }

    public void sleep() {
        System.out.println(name + " is sleeping.");
    }
}

// Child class
class Dog extends Animal {
    String breed;

    public Dog(String name, String breed) {
        super(name);    // call parent constructor
        this.breed = breed;
    }

    // Additional method
    public void bark() {
        System.out.println(name + " is barking.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog("Rex", "German Shepherd");
        myDog.eat(); // Rex is eating.  (inherited)
        myDog.sleep(); // Rex is sleeping.  (inherited)
        myDog.bark(); // Rex is barking.  (own)
        System.out.println("Breed: " + myDog.breed);
    }
}

```

Output:

text

Rex is eating.

Rex is sleeping.

Rex is barking.

Breed: German Shepherd

3. Types of Inheritance

In Java, we have **single**, **multilevel**, and **hierarchical** inheritance.

(Class-based multiple inheritance is **not** allowed.)

a) Single Inheritance

One child class extends one parent class.

```
class B extends A { }
```

b) Multilevel Inheritance

A chain of inheritance: class C extends B extends A.

```
class B extends A { }
```

```
class C extends B { }
```

c) Hierarchical Inheritance

Multiple child classes extend the same parent class.

```
class B extends A { }
```

```
class C extends A { }
```

Example of Multilevel:

```
java
```

```
class Vehicle {  
    void start() { System.out.println("Vehicle started"); }  
}
```

```
class Car extends Vehicle {  
    void honk() { System.out.println("Car honking"); }  
}
```

```
class SportsCar extends Car {  
    void turbo() { System.out.println("Turbo mode"); }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        SportsCar sc = new SportsCar();  
        sc.start(); // from Vehicle  
        sc.honk(); // from Car  
        sc.turbo(); // own  
    }  
}
```

4. Use of super Keyword

super is used to refer to the **parent class** members:

- `super(parameters)` – calls the parent constructor (must be the first statement in the child constructor).
- `super.method()` – calls a parent method (even if overridden).
- `super.field` – accesses a parent field (if hidden by child field).

Example:

```

java
class Parent {
    int value = 10;

    Parent() {
        System.out.println("Parent constructor");
    }

    void display() {
        System.out.println("Parent display");
    }
}

class Child extends Parent {
    int value = 20;

    Child() {
        super(); // calls Parent constructor
        System.out.println("Child constructor");
    }

    void show() {
        System.out.println("Child value: " + value);
        System.out.println("Parent value: " + super.value);
        super.display(); // calls Parent's display
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.show();
    }
}

```

Output:

text

Parent constructor

Child constructor

Child value: 20

Parent value: 10

Parent display

5. Aggregation (HAS-A relationship)

Aggregation is a form of **composition** where one class contains a reference to another class. It represents a **HAS-A** relationship. Unlike inheritance (IS-A), aggregation is a **weak** association – the contained object can exist independently.

Example:

```
java
class Address {
    String city, state, country;
    Address(String city, String state, String country) {
        this.city = city;
        this.state = state;
        this.country = country;
    }
}

class Employee {
    int id;
    String name;
    Address address; // HAS-A relationship

    Employee(int id, String name, Address address) {
        this.id = id;
        this.name = name;
        this.address = address;
    }

    void display() {
        System.out.println("ID: " + id + ", Name: " + name);
        System.out.println("Address: " + address.city + ", " + address.state + ", " +
address.country);
    }
}

public class Main {
    public static void main(String[] args) {
        Address addr = new Address("New York", "NY", "USA");
        Employee emp = new Employee(101, "John", addr);
        emp.display();
    }
}
```

Output:

text

ID: 101, Name: John

Address: New York, NY, USA

6. Assignment 9

Task: Create a class Shape with a method area() that returns 0.0. Create two subclasses Rectangle (with width and height) and Circle (with radius). Override the area() method in each to calculate the actual area. Use super to call the parent constructor. Also add a method displayInfo() in both subclasses that prints the dimensions and the area. Write a main to test.

Solution 9

java

```
class Shape {
    String type;

    Shape(String type) {
        this.type = type;
    }

    double area() {
        return 0.0;
    }
}

class Rectangle extends Shape {
    double width, height;

    Rectangle(double width, double height) {
        super("Rectangle");
        this.width = width;
        this.height = height;
    }

    @Override
    double area() {
        return width * height;
    }

    void displayInfo() {
        System.out.println("Type: " + type);
    }
}
```

```

        System.out.println("Width: " + width + ", Height: " + height);
        System.out.println("Area: " + area());
    }
}

```

```

class Circle extends Shape {
    double radius;

    Circle(double radius) {
        super("Circle");
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }

    void displayInfo() {
        System.out.println("Type: " + type);
        System.out.println("Radius: " + radius);
        System.out.println("Area: " + area());
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Rectangle rect = new Rectangle(5, 4);
        Circle circ = new Circle(3);

        rect.displayInfo();
        System.out.println("-----");
        circ.displayInfo();
    }
}

```

Output:

text

Type: Rectangle

Width: 5.0, Height: 4.0

Area: 20.0

Type: Circle
Radius: 3.0
Area: 28.274333882308138

Session 10 – Polymorphism

1. Understanding Polymorphism

Polymorphism means “many forms”. It allows objects of different classes to be treated as objects of a common superclass, and the same method call can behave differently based on the actual object type.

Polymorphism in Java is of two types:

- **Compile-time Polymorphism (Method Overloading)** – resolved at compile time.
 - **Runtime Polymorphism (Method Overriding)** – resolved at runtime.
-

2. How Polymorphism is Achieved

a) Method Overloading (Compile-time)

- Same method name but **different parameter lists** (number, type, or order).
- Return type may be different, but it is not sufficient for overloading.
- Occurs in the **same class** or between parent and child (but child can overload too).

Example:

```
java
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

b) Method Overriding (Runtime)

- Subclass provides a **specific implementation** of a method that is already defined in its superclass.
- Method signature (name, parameters, return type) must match (or return a covariant type).
- The overridden method is called based on the **actual object type** at runtime.
- Use `@Override` annotation (optional but recommended).

Example:

```

java
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

```

3. Runtime Polymorphism with Upcasting

When you refer to a subclass object using a superclass reference, the **overridden method** of the actual object is called.

Example:

```

java
public class Main {
    public static void main(String[] args) {
        Animal myAnimal; // superclass reference

        myAnimal = new Dog();
        myAnimal.sound(); // Dog barks

        myAnimal = new Cat();
        myAnimal.sound(); // Cat meows
    }
}

```

This is the essence of polymorphism – the same reference variable behaves differently based on the object it points to.

4. Assignment 10

Task: Create a class Vehicle with a method startEngine(). Create two subclasses: Car and Bike. Override startEngine() in each to print a specific message. Write a method testDrive(Vehicle v) that calls startEngine(). In main(), create objects of Car and Bike, pass them to testDrive(), and observe which version is called. Also demonstrate method overloading in the testDrive method (e.g., overload with a parameter int speed).

Solution 10

```
java
class Vehicle {
    void startEngine() {
        System.out.println("Vehicle engine started.");
    }
}

class Car extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Car engine started with a roar.");
    }
}

class Bike extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Bike engine started with a vroom.");
    }
}

public class Main {
    // Overloaded methods – compile-time polymorphism
    static void testDrive(Vehicle v) {
        System.out.print("Test drive: ");
        v.startEngine(); // runtime polymorphism
    }

    static void testDrive(Vehicle v, int speed) {
        System.out.print("Test drive at speed " + speed + ": ");
        v.startEngine();
    }

    public static void main(String[] args) {
        Vehicle car = new Car();
```

```

Vehicle bike = new Bike();

testDrive(car);    // Test drive: Car engine started with a roar.
testDrive(bike);   // Test drive: Bike engine started with a vroom.

testDrive(car, 120); // Test drive at speed 120: Car engine started with a roar.
}
}

```

Output:

text

Test drive: Car engine started with a roar.

Test drive: Bike engine started with a vroom.

Test drive at speed 120: Car engine started with a roar.

Session 11 – Abstract Class and Interface

1. Understanding Abstraction

Abstraction is the concept of hiding complex implementation details and showing only the essential features of an object.

In Java, abstraction is achieved through:

- **Abstract classes** – you cannot instantiate them; they can have abstract methods (without body) and concrete methods.
- **Interfaces** – a contract that specifies what a class must do, but not how.

2. How Abstraction is Achieved

a) Abstract Classes

- Declared with abstract keyword.
- Can have constructors, fields, and both abstract and concrete methods.
- If a class contains at least one abstract method, it must be declared abstract.
- Subclasses must implement all abstract methods (unless they are also abstract).

Example:

```

java
abstract class Shape {
    String color;

    Shape(String color) {
        this.color = color;
    }

    // abstract method – no body
    abstract double area();
}

```

```
// concrete method
void displayColor() {
    System.out.println("Color: " + color);
}
}
```

```
class Circle extends Shape {
    double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}
```

b) Interfaces

- Declared with interface keyword.
- All methods are **public abstract** by default (until Java 8 added default and static methods).
- Fields are **public static final** by default (constants).
- A class can implement **multiple** interfaces – this is Java’s way of multiple inheritance.
- An interface can extend another interface.

Example:

```
java
interface Drawable {
    void draw(); // public abstract
}

interface Resizable {
    void resize(double factor);
}

class Circle implements Drawable, Resizable {
    private double radius;

    Circle(double radius) {
```

```

        this.radius = radius;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a circle of radius " + radius);
    }

    @Override
    public void resize(double factor) {
        radius *= factor;
        System.out.println("Resized to radius " + radius);
    }
}

```

3. Use of Interface

Interfaces are used to:

- Define a **contract** that multiple classes can implement, regardless of their inheritance hierarchy.
- Enable **multiple inheritance** of type (a class can implement many interfaces).
- Achieve **loose coupling** – code to interfaces, not implementations.
- Provide a way to **add new functionality** to existing classes without modifying them.

4. Difference between Abstract Class and Interface

Feature	Abstract Class	Interface
Keyword	abstract	interface
Instantiation	Cannot be instantiated.	Cannot be instantiated.
Multiple inheritance	Only one abstract class can be extended.	A class can implement multiple interfaces.
Fields	Can have instance variables (non-final).	All fields are public static final (constants).
Methods	Can have both abstract and concrete methods.	All methods are public abstract (except default and static methods in 8+).

Feature	Abstract Class	Interface
Access modifiers	Can have private, protected, etc.	All methods are public by default.
Use case	When you want to share code among closely related classes.	When you want to define a common behavior unrelated classes.

5. Assignment 11

Task: Create an abstract class `Employee` with fields `name` and `id`, a constructor, and an abstract method `calculateSalary()`. Also create a concrete method `displayInfo()`. Create two subclasses: `FullTimeEmployee` (has `baseSalary` and `bonus`) and `ContractEmployee` (has `hourlyRate` and `hoursWorked`).

Implement `calculateSalary()` accordingly.

Create an interface `Bonus` with a method `applyBonus(double percentage)` and make `FullTimeEmployee` implement it (the bonus multiplies the salary).

In `main()`, create an array of `Employee` references, populate with both types, and call `calculateSalary()` and `displayInfo()` polymorphically. Also call `applyBonus()` on the `FullTimeEmployee`.

Solution 11

```
java
// Abstract class
abstract class Employee {
    protected String name;
    protected int id;

    public Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    // Abstract method
    public abstract double calculateSalary();

    // Concrete method
    public void displayInfo() {
        System.out.println("ID: " + id + ", Name: " + name);
        System.out.println("Salary: $" + calculateSalary());
    }
}
```

```

// Interface
interface Bonus {
    void applyBonus(double percentage);
}

// FullTimeEmployee subclass
class FullTimeEmployee extends Employee implements Bonus {
    private double baseSalary;
    private double bonus;

    public FullTimeEmployee(String name, int id, double baseSalary, double bonus) {
        super(name, id);
        this.baseSalary = baseSalary;
        this.bonus = bonus;
    }

    @Override
    public double calculateSalary() {
        return baseSalary + bonus;
    }

    @Override
    public void applyBonus(double percentage) {
        double increment = calculateSalary() * percentage / 100;
        baseSalary += increment; // bonus applied to base
        System.out.println("Bonus applied: " + percentage + "% -> added $" + increment);
    }
}

// ContractEmployee subclass
class ContractEmployee extends Employee {
    private double hourlyRate;
    private int hoursWorked;

    public ContractEmployee(String name, int id, double hourlyRate, int hoursWorked) {
        super(name, id);
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }
}

```

```

@Override
public double calculateSalary() {
    return hourlyRate * hoursWorked;
}
}

public class Main {
    public static void main(String[] args) {
        // Array of Employee references
        Employee[] employees = new Employee[2];
        employees[0] = new FullTimeEmployee("Alice", 101, 5000, 500);
        employees[1] = new ContractEmployee("Bob", 102, 50, 80);

        for (Employee emp : employees) {
            emp.displayInfo();
            System.out.println("-----");
        }

        // Apply bonus to FullTimeEmployee (cast to Bonus interface)
        if (employees[0] instanceof Bonus) {
            Bonus bonusable = (Bonus) employees[0];
            bonusable.applyBonus(10); // 10% bonus on current salary
            employees[0].displayInfo(); // updated salary
        }
    }
}

```

Output:

text

ID: 101, Name: Alice

Salary: \$5500.0

ID: 102, Name: Bob

Salary: \$4000.0

Bonus applied: 10% -> added \$550.0

ID: 101, Name: Alice

Salary: \$6050.0

Summary of OOP Concepts Covered

Concept	Key Points
Encapsulation	Hide data with private, expose through getters/setters.
Access Specifiers	private, default, protected, public – control visibility.
Inheritance	extends – IS-A, code reuse, super keyword.
Polymorphism	Overloading (compile-time) and overriding (runtime), upcasting.
Abstraction	Abstract classes and interfaces – hide details, define contracts.
Interfaces	Multiple inheritance of type, implements, default and static methods.
Aggregation	HAS-A relationship using object references.

Final Advice:

- Practice each concept with small programs.
- Understand the **why** behind each feature – it will help you design better systems.
- Always think about **encapsulation** – protect your data.
- Use **inheritance** for code reuse, but prefer **composition** when appropriate.
- **Polymorphism** and **abstraction** are the keys to flexible, maintainable code.

Now you have a solid foundation in Java OOP. Keep coding! 🚀